

Generativity and Dynamic Opacity for Abstract Types

Andreas Rossberg^{*}
Universität des Saarlandes
rossberg@ps.uni-sb.de

ABSTRACT

The standard formalism for explaining abstract types is existential quantification. While it provides a sufficient model for type abstraction in entirely statically typed languages, it proves to be too weak for languages enriched with forms of dynamic typing, where parametricity is violated. As an alternative approach to type abstraction that addresses this shortcoming we present a calculus for dynamic type generation. It features an explicit construct for generating new type names and relies on coercions for managing abstraction boundaries between generated types and their designated representation. Sealing is represented as a generalized form of these coercions. The calculus maintains abstractions dynamically without restricting type analysis.

Categories and Subject Descriptors

D.3.3 [Language Constructs and Features]: Abstract data types; F.3.3 [Studies of Program Constructs]: Type structure

General Terms

Languages, Theory

Keywords

abstract types, existential types, dynamic typing, generativity, opacity, encapsulation

1. INTRODUCTION

Type abstraction is an important tool for structuring programs and is a fundamental feature of most module systems. Languages like Modula [33, 4], CLU [16] and ML [17, 13] provide features for specifying abstract types, either directly

^{*}This research is funded by the Deutsche Forschungsgemeinschaft (DFG) as part of SFB 378: Ressourcenadaptive kognitive Prozesse, Project NEP: Statisch getypte Programmierungsumgebung für nebenläufige Constraints

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. To copy otherwise, to republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee.

PPDP'03, August 27–29, 2003, Uppsala, Sweden.

Copyright 2003 ACM 1-58113-705-2/03/0008 ...\$5.00.

or by means of their module systems. Generally speaking, an abstract type is declared in two parts: its *signature* and an *implementation*. The former usually allows to declare a name for the abstract type and specifies the operations available on values of that type, while the latter fixes a *representation type* for those values and defines the signature's operations accordingly. The key property is that the representation type remains private: the sole way to create or access values of the abstract type from the outside is by going through the operations listed in the signature.

For illustration purposes we will use (a subset of) the Standard ML module language [17]. In SML, an abstract type's signature can be specified by a signature declaration. Consider the common example of a complex number type:

```
signature COMPLEX =  
sig  
  type complex  
  val mk : real * real -> complex  
  val re : complex -> real  
  val im : complex -> real  
  val mul : complex * complex -> complex  
end
```

An implementation is provided by a structure declaration:

```
structure C :> COMPLEX =  
struct  
  type complex = real * real (* polar *)  
  fun mk(x,y) = (sqrt(x*x+y*y), atan2(y,x)+pi)  
  fun re(a,th) = a * cos th  
  fun im(a,th) = a * sin th  
  fun mul((a1,th1), (a2,th2)) =  
    (a1*a2, rem(th1+th2, 2*pi))  
end
```

An alternative implementation might use a cartesian representation for complex numbers. In any case, the abstraction operator `:>` hides the representation type `real * real` in the sense that, to the outside, the type `complex` is different from `real * real` — or any other type, for that matter. The operations exported through the signature are the only means to compose and decompose complex numbers.

The advantage of the encapsulation idea implemented by type abstraction is twofold. First, the use of type abstraction enforces loose coupling: client code is compelled not to depend on internals of an abstract type's representation. The implementation may thus be modified freely without breaking any existing client code, as long as the signature (and the semantics of its operations) remains the same.

Even more important is the second point: the type system guarantees that values of abstract type cannot be forged by clients. Such a guarantee is an essential prerequisite for enabling implementations to maintain invariants on their representations and their internal state. For example, our complex implementation preserves the invariant that the argument θ (`th`) of the complex number is always normalized to $\theta \in [0; 2\pi[$. Type abstraction also prevents mixing values stemming from different (incompatible) implementations of the same signature, e.g. a polar and a cartesian representation of complex numbers.

In their classic paper, Mitchell and Plotkin showed that abstract types can be formalised naturally as existential types [19], using the standard typing rules as found in constructive logic (e.g. System F [8]). We will review existential types and their relation to abstract types in section 2.

1.1 Dynamic type analysis

Constructs for (dynamic) type analysis have been formulated in different flavours. Examples are dynamics [1, 14] and intensional type analysis [11] and extensional polymorphism [7]. They have in common that there is some form of typecase expression that allows branching dependent on a type that is determined dynamically.

Let us consider an extension of SML with typecase. In order to simplify the presentation, we use a very simple variant throughout this paper. Our typecase does not bind any type variables, but merely allows the type of an expression to be compared to a second type:

```
typecase exp1 :  $\tau_1$  of x :  $\tau_2$  then exp2 else exp3
```

The intuitive semantics of this expression form is that it evaluates to $exp_2[x := exp_1]$ iff $\tau_1 = \tau_2$ dynamically, to exp_3 otherwise. That semantics will be made more precise in section 2.3. An example for using a typecase might be a simplistic polymorphic string conversion function:

```
fun 'a toString (x : 'a) =
  typecase x : 'a of x' : int
  then Int.toString x'
  else typecase x : 'a of x' : real
  then Real.toString x'
  else typecase x : 'a of x' : bool
  then Bool.toString x'
  else "_"
```

By applying this function to some arbitrary value v , the polymorphic type variable `'a` will be instantiated to a concrete (dynamic) type τ , the type of v . The function will properly dispatch on that type and delegate the conversion task to a suitable library function, if available.

How does typecase interact with type abstraction? What happens, if we try to evaluate the following expression:

```
typecase C.mk(0.0, 1.0) : C.complex
of p : real * real
then print("theta = " ^ Real.toString(#2 p))
else raise CouldntAccessRepresentation
```

Or even more critical:

```
typecase (1.0, 1001.0*pi) : real * real
of z : C.complex
then z
else raise CouldntAccessRepresentation
```

It is obvious that in both cases the `else` branch should be chosen. Or is it? Unfortunately, this is not the answer the standard model of abstract types using existential quantification will give! The reasons will become apparent in section 2.3. In fact, it is well-known that existential abstraction can be broken in the presence of primitives for type analysis, because the presence of the latter causes loss of the parametricity property [30, 26] its encapsulation power relies on. Weirich demonstrated that in a non-parametric setting arbitrary values of existential type can be cast back and forth to and from their actual representation type [32]. While such a cast is still type-safe in the sense of not violating soundness, it clearly undermines any of the previously mentioned guarantees the type system should make about abstract types — the first expression above is coupled to internals of the complex representation, while the second even breaks its invariant on the complex angle θ . Because type abstraction is no longer sufficient to ensure encapsulation, it is practically rendered useless.

1.2 Agenda

How can the conflict be solved? A simple possibility is to forbid analysis of abstract types altogether. For example, Harper and Morrisett curiously propose to distinguish between analyzable and non-analyzable types [11]. However, this clearly is overly restrictive. For example, it would disallow us to extend the string conversion function to handle complex numbers, by rendering the following code illegal:

```
fun 'a toString (x : 'a) =
  typecase x : 'a of x' : C.complex then
    Real.toString(C.re x') ^
    (if im x' >= 0.0 then "+" else "-") ^
    Real.toString(abs(C.im x')) ^ "i"
  else ...
```

Similarly, in a language with type `dynamic`, it would become impossible to inject values with abstract type into `dynamic` — or more precisely, to project them out again. Hence, such a solution might seriously impair the usefulness of type analysis as well as the applicability of type abstraction.

This paper thus aims to define a formal semantics for type abstraction that is fully compatible with type analysis. In short, we seek a semantics in which the interplay between both features has the following characteristics:

1. *dynamic opacity*: an abstract type cannot be identified with any other type through dynamic analysis,
2. *full reflexivity*: every type can be analyzed.

Dynamic opacity basically says that the key property of type abstraction ought to carry over from the static type system to dynamic typing: abstract types need to be inaccessible and unforgeable even by means of dynamic type analysis. The second property effectively means that any type must be comparable (dynamically) to any other. We borrow the term *full reflexivity* from Trifonov, Saha and Shao [31], who introduced it in a slightly different context to express the absence of any restriction on the *syntactic form* of types that are available for analysis (no such restriction is necessary for the weak typecase used here). Taken together, both requirements imply that an abstract type must be different from any other type in the language's universe of types. It clearly follows that type abstraction must have some sort of *generative* operational semantics: introduction of an abstract type

dynamically generates a new type. Without generativity, type abstraction has no dynamic interpretation!

It should be noted that we solely discuss the requirements of dynamic typing intended for programmatic use, i.e. as a language feature available to the programmer in the external language. There are different application domains for type analysis, especially in language implementations for dealing with specialised data representations in the compilation of polymorphic functions (which was the motivation for Harper and Morrisett's work). Such internal use demands for different, incompatible properties. In particular, dynamic opacity is specifically not wanted under such circumstances. We view external and internal use of dynamic typing as largely independent issues, and will not further consider the latter.

1.3 Plan

In section 2 we give a short overview over abstract types in the existential type encoding and investigate how it interferes with dynamic type analysis. In section 3 we introduce the basic features of the λ_N -calculus, which we propose as an alternative model. Section 4 discusses the calculus and its basic properties formally. In section 5 we look at its higher-order generalization and an extension incorporating an alternative, applicative notion of generativity. We review some related work in section 6 and conclude in section 7.

2. ABSTRACTION BY EXISTENTIAL QUANTIFICATION

In this section we will give a short recap of existential types and their correspondence to abstract types. We then discuss in more detail how this correspondence is destroyed by adding dynamic type analysis. We write $\equiv$ for syntactic equivalence (modulo α -conversion). $FV(e)$ denotes the set of free term variables of e defined in the usual way, and $FTV(t)$ the free type variables of type or term t . For clarity, we will sometimes use the notation (let $x = e_1$ in e_2) as an abbreviation for the expression $(\lambda x:\tau_1.e_2) e_1$, where τ_1 is the type of e_1 . Moreover, we sometimes use $_-$ for *don't care* variables.

2.1 Existential types

Figure 1 shows the syntax, evaluation and typing rules for existential types, as an extension to the plain polymorphic lambda calculus [3]. A value of existential type is usually called a *package*. It essentially is a pair $\langle \tau, e \rangle$, encapsulating a *representation type* τ and an *implementation* e . A package is assigned existential type $\exists \alpha.\tau'$ if its implementation matches the *signature type* τ' , by replacing all occurrences of α with the actual representation type τ . Unfortunately, the signature type is not determined uniquely by the implementation and representation types alone, thus it has to be annotated explicitly, as apparent from the syntax.¹

In order to do anything interesting with a package, i.e. access the encapsulated implementation, the existential quantifier has to be eliminated. In the expression form $(\text{open } \langle \alpha, x \rangle = e_1 \text{ in } e_2)$ the subexpression e_1 denotes a package, whose representation type and implementation can be referred to through variables α and x within e_2 , respectively. Such an open expression evaluates to its body e_2 , by substituting its

¹Another common syntax for existential introduction is $(\text{pack } \tau_1, e \text{ as } \exists \alpha.\tau_2)$ and variants thereof.

(types) $\tau ::= \dots \mid \exists \alpha.\tau$
 (terms) $e ::= \dots \mid \langle \tau, e \rangle : \exists \alpha.\tau' \mid \text{open } \langle \alpha, x \rangle = e_1 \text{ in } e_2$

$\text{open } \langle \alpha, x \rangle = \langle \tau, e_1 \rangle : \exists \alpha.\tau' \text{ in } e_2 \rightarrow e_2[\alpha := \tau][x := e_1]$

(PACK) $\frac{\Gamma \vdash e : \tau'[\alpha := \tau]}{\Gamma \vdash (\langle \tau, e \rangle : \exists \alpha.\tau') : \exists \alpha.\tau'}$

(OPEN) $\frac{\Gamma \vdash e_1 : \exists \alpha.\tau' \quad \Gamma, \alpha, x:\tau' \vdash e_2 : \tau \quad (\alpha \notin FV(\tau))}{\Gamma \vdash (\text{open } \langle \alpha, x \rangle = e_1 \text{ in } e_2) : \tau}$

Figure 1: Existential types

bound variables with the actual type and value found in the package.²

2.2 Encoding Abstract Types

An abstract type declaration introduces a new type bundled with a set of operations available on values of that type. An encoding via existential types is relatively straightforward. Let us assume that the polymorphic lambda calculus has been enriched further with product types and real numbers. Then the SML signature **COMPLEX** from the introduction can be represented by the type

$\text{COMPLEX} \equiv \exists \alpha.(\text{real} \times \text{real} \rightarrow \alpha) \times (\alpha \rightarrow \text{real}) \times (\alpha \rightarrow \text{real}) \times (\alpha \times \alpha \rightarrow \alpha)$

That is, the set of operations is mapped to a tuple of appropriate type, and this type is existentially quantified over the type to be hidden by the abstraction. The structure **C** can be modelled as (taking the freedom to use tuple patterns):

$C \equiv \langle \text{real} \times \text{real},$
 $(\lambda(x, y) : \text{real} \times \text{real} . (\sqrt{x^2 + y^2}, \arctan(y/x) + \pi),$
 $\lambda(a, \theta) : \text{real} \times \text{real} . a \cdot \cos \theta,$
 $\lambda(a, \theta) : \text{real} \times \text{real} . a \cdot \sin \theta,$
 $\lambda((a_1, \theta_1), (a_2, \theta_2)) : (\text{real} \times \text{real}) \times (\text{real} \times \text{real}) .$
 $(a_1 \cdot a_2, \text{rem}(\theta_1 + \theta_2, 2\pi))$
 $\rangle : \text{COMPLEX}$

ML style module access does not map as directly, because structure components are accessed using the dot notation, while a package has to be opened explicitly to make its content available. However, Cardelli and Leroy have shown that there exists a systematic translation from dot notation into plain existential types [5]. Essentially, the package encoding a structure is opened immediately. For our purpose,

$\text{val } a = \text{C.re}(\text{C.mk}(0.0, 1.0))$

might be encoded as

$a \equiv \text{open } \langle \alpha, (mk, re, -, -) \rangle = C \text{ in } re \ (mk \ (0, 1))$

The typing rules for open plus the standard hygiene conventions for bound variables ensure that α is distinct from any other type variable in the same scope and thus behaves like

²Often the existential elimination form is written $(\text{unpack } e_1 \text{ as } \alpha, x \text{ in } e_2)$.

$$\begin{aligned}
e &::= \dots \mid \text{tcase } e_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } e_3 \\
(\text{TCase}) \quad &\frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash \tau_2 : \Omega \quad \Gamma, x : \tau_2 \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{tcase } e_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } e_3) : \tau} \\
&\text{tcase } e_1 : \tau \text{ of } x : \tau \text{ then } e_2 \text{ else } e_3 \rightarrow e_2[x := e_1] \\
&\text{tcase } e_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } e_3 \rightarrow e_3 \quad (\tau_1 \neq \tau_2)
\end{aligned}$$

Figure 2: A typecase extension

a “fresh” type. Moreover, it behaves fully abstract because in standard λ -calculi every expression is *parametric* [25] in all type variables, meaning that reduction can proceed uniformly for all possible instantiations. In particular, the body e_2 of an open expression is parametric with respect to the bound variable α — evaluation will never depend on the actual representation type τ of the package being opened, although α is substituted by τ during reduction. That key observation establishes the close correspondence between existential types and abstract types.

2.3 Interaction with dynamic type analysis

Figure 2 specifies the semantics of our typecase, as an extension to the lambda calculus.³ It provides only a simple form of type analysis but suffices to demonstrate the fundamental problem.

We have seen that the encoding of abstract types via existentials crucially relies on the parametricity property. That property breaks down in the face of operations for type analysis: if a polymorphic function is able to analyse its type argument using typecase, it obviously will not evaluate independently of any concrete instantiation. Similarly, a function that is passed an argument of existentially quantified type might inspect the type encapsulated by the quantifier — the computation can be dependent on the actual representation type. Recall the typecase expression from section 1.1 that was incriminated to break the complex invariant. Expressed with existential types (and $\perp$) it may look like follows:

$$\begin{aligned}
\text{open } \langle \alpha, (mk, re, im, mul) \rangle &= \langle real \times real, \dots \rangle \text{ in} \\
(\dots \text{ tcase } (1, 1001\pi) : real \times real \text{ of } z : \alpha &\text{ then } z \text{ else } \perp \dots)
\end{aligned}$$

But upon reduction of the open expression the type variable α naming the abstract type will be substituted and reveal

$$\rightarrow (\dots \text{ typecase } (1, 1001\pi) : real \times real \text{ of } z : real \times real \text{ then } z \text{ else } \perp \dots)$$

Both types in the typecase are now equal and the construct returns $z \equiv (1, 1001\pi)$ from its left branch having the same static type α as proper complex values.

³Adding typecase to the polymorphic lambda calculus without restricting the reduction relation breaks confluence. Consider $(\lambda\alpha.\lambda x:\alpha.\text{tcase } x:\alpha \text{ of } y:\text{int} \text{ then } 1 \text{ else } 0) \text{ int } 9$. Depending on which redex gets reduced first, this expression yields 1 or 0. For simplicity, we hence assume a call-by-value strategy.

3. TOWARDS A FORMAL SEMANTICS FOR DYNAMIC TYPE GENERATION

Although well known, the interference between existential types and type analysis has received only little attention in prior work. Besides the proposal by Harper and Morrisett mentioned in the introduction, Abadi, Cardelli, Pierce and Rémy [1] already suggested generativity as a solution, observing that dynamic opacity can be achieved by simply replacing the type variable bound by open with a “fresh” type constant during evaluation. Their idea amounts to changing the corresponding reduction rule to:

$$\text{open } \langle \alpha, x \rangle = \langle \tau, e_1 \rangle : \exists \alpha. \tau' \text{ in } e_2 \rightarrow e_2[\alpha := t][x := e_1]$$

where t is a fresh type constant. Obviously, the representation type could no longer be analysed transparently. Unfortunately, this modification destroys type preservation, as can easily be seen from the following example, which is a simple η -expansion of the expression a given in section 2.2:

$$a' \equiv \text{open } \langle \alpha, (mk, re, -, -) \rangle = C \text{ in } (\lambda z:\alpha. re \ z) (mk \ (0, 1))$$

This term is well-typed, but after applying the above reduction rule it becomes:

$$(\lambda z:t. (\lambda(a, \theta) : real \times real. a \cdot \cos \theta) \ z) ((\lambda \dots) (0, 1))$$

which is no longer typable — there is a clash between the abstract type t and its respective representation type $real \times real$, which is the argument type of function re .

Consequently, in order to make the idea of using generativity for abstraction work, we have to solve two problems: the concept of dynamic type “freshness” must be fleshed out formally, and transitions between abstract and concrete type must be managed in a sound way.

3.1 Generativity

Formalisms for describing dynamic generation of fresh *value* names are well developed. For example, the name restriction form $\nu n. P$ is a central feature of the π -calculus [28] and can be viewed as an expression that generates a new name n with local scope. Pitts’ λ_ν -calculus [24] transfers that idea to the λ -calculus, although with a different implementation.

We introduce a similar construct, but for generating *type* names instead of value names. We will use the notation

$$N\gamma \approx_{\tau} e$$

(with N read as upper-case nu) to introduce a fresh type name γ within expression e . N -bound names are subject to standard α -conversion rules. Because having a fresh type that is not inhabited by any values is not very interesting on its own, the N -form also declares a representation type τ . Within e the relation between the new type and its representation is known and can be used to construct and inspect values of type γ . Outside the scope of the corresponding N -expression that relation is not visible. We defer the discussion on how values are constructed to the next section.

How do we track generated type names? We chose to take the path of the π -calculus, where ν -expressions never get eliminated, but float outwards by special equivalence rules for *scope extrusion*.⁴ In order to allow interaction between

⁴We also considered the alternative approach of introducing an explicit type store or heap as in the λ_ν -calculus, but that choice would produce a more complicated system.

a N-expression's body e and the expression's context, we incorporate reduction rules in a similar spirit, e.g.:

$$\begin{array}{lll} (N\gamma \approx \tau. e_1) e_2 & \rightarrow & N\gamma \approx \tau. (e_1 e_2) \quad (\gamma \notin \text{FTN}(e_2)) \\ e_1 (N\gamma \approx \tau. e_2) & \rightarrow & N\gamma \approx \tau. (e_1 e_2) \quad (\gamma \notin \text{FTN}(e_1)) \end{array}$$

The side conditions on the free type names of subterms (written $\text{FTN}(e)$) ensure that the context cannot capture the type name γ .

The typing rule for N is straight-forward:

$$\frac{\Gamma, \gamma \approx \tau \vdash e : \tau'}{\Gamma \vdash N\gamma \approx \tau. e : \tau'} (\gamma \notin \text{FTN}(\tau'))$$

We need to be able to record the *type assertion* $\gamma \approx \tau$ in the environment, so that γ is properly related to its representation. The side condition keeps γ from escaping its scope, and is similar to the side condition of the (OPEN) typing rule for existential types.

3.2 Coercions

What does it mean for a new type γ to be 'represented' by τ ? It certainly cannot mean that both types are simply equivalent (i.e. $\gamma = \tau$), since such an interpretation would not make γ particularly 'new' and inevitably bring us back to a semantics that violates dynamic opacity. We always have to be able to distinguish both types. Consequently, in order to avoid running into type preservation problems, we also need to be able to distinguish values of both types. Hence we require appropriate coercions to go from the abstract type to its representation and vice versa. We will use the notation

$$\{e\}_{\gamma}^{+}$$

for coercing a value e of representation type τ to the abstract type γ . Dually, we have

$$\{e'\}_{\gamma}^{-}$$

for the inverse coercion. Coercions allow an implementation to perform appropriate type conversions for any value of abstract type γ that crosses the abstraction boundaries in either direction. Positive coercions can be seen as constructors for values of the new type. They are eliminated only by negative coercions, the corresponding destructors. Consequently, the only evaluation possible with coercions is *cancellation*, implemented by a single reduction rule:

$$\{\{e\}_{\gamma}^{+}\}_{\gamma}^{-} \rightarrow e$$

The standard scoping rules guarantee that only coercions belonging to the same abstract type can cancel out each other.

The typing rules for coercions are obvious:

$$\frac{\Gamma \vdash e : \tau \quad \gamma \approx \tau \in \Gamma}{\Gamma \vdash \{e\}_{\gamma}^{+} : \gamma} \quad \frac{\Gamma \vdash e : \gamma \quad \gamma \approx \tau \in \Gamma}{\Gamma \vdash \{e\}_{\gamma}^{-} : \tau}$$

Due to the side conditions in the rules, coercions are only available within the lexical scope of the corresponding type generator, thus the transition across abstraction boundaries can only be triggered from within the abstraction.

3.3 Example

Recall the *complex* example from the introduction and its encoding with existential types. Rewritten using type

generation it looks as follows:

$$\begin{aligned} C' &\equiv N\gamma \approx \text{real} \times \text{real}. \\ &\langle \gamma, \\ &\quad (\lambda(x, y) : \text{real} \times \text{real} . \{(\sqrt{x^2 + y^2}, \arctan(y/x) + \pi)\}_{\gamma}^{+}, \\ &\quad \lambda z : \gamma . \text{let } (a, \theta) = \{z\}_{\gamma}^{-} \text{ in } a \cdot \cos \theta, \\ &\quad \lambda z : \gamma . \text{let } (a, \theta) = \{z\}_{\gamma}^{-} \text{ in } a \cdot \sin \theta, \\ &\quad \lambda(z_1, z_2) : \gamma \times \gamma . \text{let } (a_1, \theta_1) = \{z_1\}_{\gamma}^{-} \text{ in} \\ &\quad \quad \text{let } (a_2, \theta_2) = \{z_2\}_{\gamma}^{-} \text{ in} \\ &\quad \quad \{(a_1 \cdot a_2, \text{rem}(\theta_1 + \theta_2, 2\pi))\}_{\gamma}^{+} \\ &\quad) \rangle : \text{COMPLEX} \end{aligned}$$

We still use an existential type. However, it is no longer utilized for providing abstraction, but merely for closing the signature of the abstraction (recall that γ itself may not appear in the signature). By putting the abstract type into a package it becomes accessible from the outside. The signature of the abstraction is uniquely determined by the uses of γ and corresponding coercions in its implementation. Hence *COMPLEX* is the only possible type for the package.

The rewritten abstract type C' can be used as before, via simply opening the package:

$$\text{open } \langle \alpha, (mk, re, _, _) \rangle = C' \text{ in } re (mk (0, 1))$$

The contained N-binder will be shifted outwards automatically by the corresponding scope extrusion rules.

3.4 A-posteriori abstraction

So far, to build an abstraction its implementation has to use coercions internally, in order to meet the intended signature type. We speak of a *a priori* abstraction: an implementation must be tailored to a particular signature. On the other hand, abstraction based on existential types happens *a posteriori*: arbitrary parts of a given implementation's type are just hidden away without affecting the implementation itself, a construction sometimes called *sealing* in the context of modules [6]. Can we recover that flexibility?

The answer is yes: given an abstract signature type and a suitable implementation, we can systematically construct an expression that coerces the whole implementation into the desired signature type. Let e be an expression that shall be sealed with signature τ' , abstracting away some type τ as γ . Assuming $e : \tau'[\gamma := \tau]$, the term produced by applying the transformation $\{e : \tau'\}_{\gamma \approx \tau}^{+}$ defined in figure 3 will have the desired signature type. The transformation is defined inductively over the signature type τ' . It constructs an η -expansion of the original term e , wrapping all parts e' that are supposed to get type γ into a suitable coercion $\{e'\}_{\gamma}^{+}$. Function types require an inverse treatment of their argument, where $e' : \gamma$ is replaced by $\{e'\}_{\gamma}^{-}$ instead. Since the inverse transformation is completely dual, we use the notation $\pm$ to capture both directions in a single set of rules. Coercion polarity is inverted for function arguments. The following lemma captures the central invariants:

LEMMA 1 (A-POSTERIORI ABSTRACTION INVARIANTS).
Let e be an expression and Γ an environment with $\gamma \approx \tau \in \Gamma$.

1. If $\Gamma \vdash e : \tau'[\gamma := \tau]$, then $\Gamma \vdash \{e : \tau'\}_{\gamma \approx \tau}^{+} : \tau'$.
2. If $\Gamma \vdash e : \tau'$, then $\Gamma \vdash \{e : \tau'\}_{\gamma \approx \tau}^{-} : \tau'[\gamma := \tau]$.

$$\begin{aligned}
\{e : \gamma\}_{\gamma \approx \tau}^{\pm} &= \{e\}_{\gamma}^{\pm} \\
\{e : \gamma'\}_{\gamma \approx \tau}^{\pm} &= e & (\gamma' \neq \gamma) \\
\{e : \alpha\}_{\gamma \approx \tau}^{\pm} &= e \\
\{e : \tau_1 \rightarrow \tau_2\}_{\gamma \approx \tau}^{\pm} &= \lambda x : \{ \tau_1 \}_{\gamma \approx \tau}^{\pm} . \{e : x : \tau_1\}_{\gamma \approx \tau}^{\mp} : \tau_2\}_{\gamma \approx \tau}^{\pm} \\
\{e : \forall \alpha. \tau_1\}_{\gamma \approx \tau}^{\pm} &= \Lambda \alpha. \{e : \alpha : \tau_1\}_{\gamma \approx \tau}^{\pm} \\
\{\tau'\}_{\gamma \approx \tau}^+ &= \tau' \\
\{\tau'\}_{\gamma \approx \tau}^- &= \tau'[\gamma := \tau]
\end{aligned}$$

Figure 3: A-posteriori abstraction

With this in mind we introduce another building block of our calculus: in order to allow it to express sealing directly, we generalize coercion expressions to arbitrary types and make the transformation rules from figure 3 built-in by turning them into reduction rules. That is, coercions will have the actual form

$$\{e : \tau'\}_{\gamma \approx \tau}^{\pm}$$

In this generalized form, coercions are reminiscent of the abstraction brackets by Grossman, Morrisett and Zdancewic [10]. We will discuss that connection in section 6. Using generalized coercions, first-class abstract types can be represented by expressions of the form

$$N\gamma \approx \tau. \langle \gamma, \{e : \tau'\}_{\gamma \approx \tau}^+ \rangle$$

where τ is the representation type, τ' the signature, and e the implementation of the abstract type.

3.5 Polymorphic coercions

We have not yet given the typing rules for generalized coercions. The lemma from the previous section suggests

$$\frac{\Gamma \vdash e : \tau'[\gamma := \tau] \quad \gamma \approx \tau \in \Gamma}{\Gamma \vdash \{e : \tau'\}_{\gamma \approx \tau}^+ : \tau'} \quad \frac{\Gamma \vdash e : \tau' \quad \gamma \approx \tau \in \Gamma}{\Gamma \vdash \{e : \tau'\}_{\gamma \approx \tau}^- : \tau'[\gamma := \tau]}$$

Obviously, these rules generalize the typing rules for simple coercions, for $\{e\}_{\gamma}^{\pm} = \{e : \gamma\}_{\gamma \approx \tau}^{\pm}$. Unfortunately, they are not correct. First note that it is not possible to seal a value twice with respect to the same type γ : in the sealing rule, the type of e must be free of γ . Dually, unsealing always delivers a γ -free type. Consequently, a complication arises with *polymorphic coercions*. Consider the following term, for example:

$$P \equiv (\Lambda \alpha. \lambda x : \alpha. \{x : \alpha\}_{\gamma \approx \tau}^-) \gamma$$

Under the above rules, P would be assigned type $\gamma \rightarrow \gamma$. However, standard β -reduction yields

$$\lambda x : \gamma. \{x : \gamma\}_{\gamma \approx \tau}^-$$

which has type $\gamma \rightarrow \tau$. Type preservation is violated. The problem is, that turned into reduction steps, the abstraction transformation is not static but interleaved with other reductions. Hence, the typing rules must account for potential substitutions. Intuitively, the γ -substitutions in the coercion typing rules must be delayed until all free type variables in τ' have been substituted. We deal with this by introducing *unsealed types* of the form

$$\{\tau'\}_{\gamma \approx \tau}^-$$

that essentially perform a substitution on τ' , as the following set of special equivalence rules reveals:

$$\begin{aligned}
\{\gamma\}_{\gamma \approx \tau}^- &= \tau \\
\{\gamma'\}_{\gamma \approx \tau}^- &= \gamma' & (\gamma' \neq \gamma) \\
\{\tau_1 \rightarrow \tau_2\}_{\gamma \approx \tau}^- &= \{\tau_1\}_{\gamma \approx \tau}^- \rightarrow \{\tau_2\}_{\gamma \approx \tau}^- \\
\{\forall \alpha. \tau_1\}_{\gamma \approx \tau}^- &= \forall \alpha. \{\tau_1\}_{\gamma \approx \tau}^-
\end{aligned}$$

There is no equivalence rule for type variables, so that a type of the form $\{\alpha\}_{\gamma \approx \tau}^-$ maintains the pending substitution for γ until α is substituted. Using this setup, sound typing rules can be given for coercions:

$$\frac{\Gamma \vdash e : \{\tau'\}_{\gamma \approx \tau}^- \quad \gamma \approx \tau \in \Gamma}{\Gamma \vdash \{e : \tau'\}_{\gamma \approx \tau}^+ : \tau'} \quad \frac{\Gamma \vdash e : \tau' \quad \gamma \approx \tau \in \Gamma}{\Gamma \vdash \{e : \tau'\}_{\gamma \approx \tau}^- : \{\tau'\}_{\gamma \approx \tau}^-}$$

As long as no type variables occur in τ' , they are equivalent to the rules above under the stated type equivalence. However, for the term P they correctly allow derivation of the type $\gamma \rightarrow \{\gamma\}_{\gamma \approx \tau}^- = \gamma \rightarrow \tau$. Likewise, they prohibit double sealing even in polymorphic cases. That implies that a polymorphic sealing function like

$$\Lambda \alpha. \lambda x : \alpha. \{x : \alpha\}_{\gamma \approx \tau}^+$$

is not well-typed. It must be formulated as either

$$\Lambda \alpha. \lambda x : \{\alpha\}_{\gamma \approx \tau}^- . \{x : \alpha\}_{\gamma \approx \tau}^+ : \forall \alpha. \{\alpha\}_{\gamma \approx \tau}^- \rightarrow \alpha$$

or

$$\Lambda \alpha. \lambda x : \alpha. \{\{x : \alpha\}_{\gamma \approx \tau}^- : \alpha\}_{\gamma \approx \tau}^+ : \forall \alpha. \alpha \rightarrow \alpha$$

In the former version, the function is not applicable to arguments that already contain occurrences of abstract type γ . The latter version lifts this restriction by simply unsealing any such potential values first.

4. THE λ_N -CALCULUS

We are now prepared to look at the calculus that we will refer to as λ_N as a whole.

4.1 Syntax

The syntax of the λ_N -calculus is shown in figure 4. Essentially, it is a polymorphic lambda calculus with recursive functions, extended with the constructs introduced in the previous section. It also contains a typecase expression, so that it allows discussion of the issues raised by dynamic typing. Values are defined in the usual way, but include abstract values of the form $\{\hat{e} : \tau\}_{\gamma \approx \tau'}^+$ with the side condition $\tau = \gamma$. Further, we distinguish a second subclass of terms called *results*, which is necessary to formulate deterministic evaluation rules for scope extrusion.

We will write $\lambda x : \tau. e$ for $(\text{fix } x' (x : \tau) : \tau'. e)$ if $x' \notin \text{FV}(e)$ and τ' is the (unique) type of e . We also abbreviate $\{e : \gamma\}_{\gamma \approx \tau}^{\pm}$ to $\{e\}_{\gamma}^{\pm}$ if τ is clear from context. We will use notation for existential types in some examples, which can be encoded in λ_N using universal types in the usual way [20]. Also, we sometimes silently assume additional types like *int*, *real*, *unit* and respective constants, or cartesian products.

Environments are extended to include type assertions $\gamma \approx \tau$ for type names. They track the validity of coercions. We take the liberty to treat environments as sets of the contained assignments and assertions, or as finite functions mapping variables to types. We write $\text{Dom}(\Gamma)$ to denote the set of all names and variables bound by Γ .

(types)	$\tau ::= \alpha \mid \gamma \mid \tau_1 \rightarrow \tau_2 \mid \forall \alpha. \tau \mid \{\tau\}_{\gamma \approx \tau'}^-$
(terms)	$e ::= x \mid \text{fix } x_1(x_2:\tau_2):\tau_1.e \mid e_1 e_2 \mid \Lambda \alpha.e \mid e \tau \mid$ $N\gamma \approx \tau.e \mid \{e:\tau\}_{\gamma \approx \tau'}^\pm \mid$ $\text{tcase } e_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } e_3$
(results)	$\hat{e} ::= \hat{e} \mid N\gamma \approx \tau.\hat{e}$
(values)	$\hat{e} ::= \text{fix } x_1(x_2:\tau_2):\tau_1.e \mid \Lambda \alpha.e \mid \{\hat{e}:\tau\}_{\gamma \approx \tau'}^+ \ (\tau = \gamma)$
(env's)	$\Gamma ::= \cdot \mid \Gamma, x:\tau \mid \Gamma, \alpha \mid \Gamma, \gamma \approx \tau$

Figure 4: λ_N Syntax

4.2 Reduction

Figure 5 collects the one-step evaluation rules for λ_N . They can be categorized into five groups: standard β -reduction rules (1–2), coercion rules (3–6), type analysis rules (7–8), scope extrusion (9–13), and structural rules (14–19). Together, they specify a deterministic call-by-value evaluation strategy. We will write $=$ for convertibility with respect to the corresponding equivalence relation generated from $\rightarrow$.

Note that the coercion rules 4–6 are overloaded for both polarities. At function type we use the following definition for substituting type annotations depending on polarity:

$$\{\tau\}_{\gamma \approx \tau'}^+ = \tau$$

Reduction of coercions and typecase is type-directed. The equivalence relation on types will be given in the next section. In rules 5 and 6 we implicitly require the equivalent types to be well-formed, i.e. $\vdash \tau_2 \rightarrow \tau_1 : \Omega$ and $\vdash \forall \alpha. \tau_1 : \Omega$, respectively. For deterministic scope extrusion we have introduced the syntactic class of *results*. A result is a value prefixed by a sequence of N-binders. Scope extrusion only applies to the outermost binder of a result; evaluation has to proceed under a N-binder until its body has become a result. Intuitively, a result may be thought of as an expression's “return value” paired with the *heap* of type names its evaluation allocated. The type names generated in different subexpressions will all accumulate in the heap of the complete expressions's result. For example, consider the following reduction sequence:

$$\begin{aligned}
& (\lambda f: \text{int} \rightarrow \text{int}. f (N\gamma_1 \approx \tau_1. f 4)) (\lambda x: \text{int}. N\gamma_2 \approx \tau_2. x) \\
\rightarrow & (\lambda x: \text{int}. N\gamma_2 \approx \tau_2. x) (N\gamma_1 \approx \tau_1. (\lambda x: \text{int}. N\gamma_2 \approx \tau_2. x) 4) \\
\rightarrow & (\lambda x: \text{int}. N\gamma_2 \approx \tau_2. x) (N\gamma_1 \approx \tau_1. N\gamma_2' \approx \tau_2. 4) \\
\rightarrow & N\gamma_1 \approx \tau_1. (\lambda x: \text{int}. N\gamma_2 \approx \tau_2. x) (N\gamma_2' \approx \tau_2. 4) \\
\rightarrow & N\gamma_1 \approx \tau_1. N\gamma_2' \approx \tau_2. (\lambda x: \text{int}. N\gamma_2 \approx \tau_2. x) 4 \\
\rightarrow & N\gamma_1 \approx \tau_1. N\gamma_2' \approx \tau_2. N\gamma_2 \approx \tau_2. 4
\end{aligned}$$

Generation is fully dynamic, i.e. the number of type names generated is not determined statically. The following non-terminating expression will actually generate an infinite number of types:

$$(\text{fix } f(x:\text{unit}):\text{unit}. N\gamma \approx \tau. f x) ()$$

4.3 Typing

The typing and well-formedness rules of the calculus are a simple extension of the rules for the polymorphic λ -calculus. We need the usual three judgment forms:

$\vdash \Gamma : \diamond$	well-formedness of environments
$\Gamma \vdash \tau : \Omega$	well-formedness of types
$\Gamma \vdash e : \tau$	well-typedness of terms

In comparison to the plain λ -calculus, λ_N adds the previously discussed typing rules for N, coercions, and typecase, as well as extended well-formedness rules for dealing with type names and type assertions. The presence of a non-trivial type equivalence relation requires an additional structural typing rule (EQUIV) for assigning of equivalent types to a term. The type equivalence relation is defined in figure 7.

It is not difficult to show that type soundness properties hold for λ_N :

THEOREM 1 (PRESERVATION). *If $\Gamma \vdash e : \tau$ and $e \rightarrow e'$, then $\Gamma \vdash e' : \tau$.*

THEOREM 2 (PROGRESS). *If $\cdot \vdash e : \tau$ (i.e. e is closed), then either $e \equiv \hat{e}$ for some result $\hat{e}$, or $e \rightarrow e'$ for some e' .*

Note that if $\hat{e}$ is closed, then $\hat{e} \not\equiv \{\hat{e}'\}_{\gamma}^+$. We also have

THEOREM 3 (UNIQUE TYPES). *Whenever $\Gamma \vdash e : \tau$ and $\Gamma \vdash e : \tau'$ then $\tau = \tau'$.*

4.4 Opacity

To see how opacity is still preserved in the non-parametric setting of λ_N let us go back to the λ_N -encoding of complex numbers, as shown in section 3.3. It is safe with respect to dynamic typing, as the reduction of the expression representing the second offending example from section 1.1 shows:

$$\begin{aligned}
& \text{open } \langle \alpha, _ \rangle = N\gamma \approx \text{real} \times \text{real} . \langle \gamma, \dots \rangle \text{ in} \\
& (\dots \text{tcase } (1, 1001\pi) : \text{real} \times \text{real} \text{ of } z : \alpha \text{ then } z \text{ else } \perp \dots) \\
\rightarrow & N\gamma \approx \text{real} \times \text{real} . \text{open } \langle \alpha, _ \rangle = \langle \gamma, \dots \rangle \text{ in} \\
& (\dots \text{tcase } (1, 1001\pi) : \text{real} \times \text{real} \text{ of } z : \alpha \text{ then } z \text{ else } \perp \dots) \\
\rightarrow & N\gamma \approx \text{real} \times \text{real} . \\
& (\dots \text{tcase } (1, 1001\pi) : \text{real} \times \text{real} \text{ of } z : \gamma \text{ then } z \text{ else } \perp \dots) \\
\rightarrow & N\gamma \approx \text{real} \times \text{real} . (\dots \perp \dots)
\end{aligned}$$

The variable z keeps an abstract type even after opening the package, and the attempt to violate the abstraction via typecase remains unsuccessful.

More generally, consider a closed, well-typed function of the form $\Lambda \alpha. \lambda x:\alpha. e$ (which may contain random uses of typecase). Applied to an abstract type γ and value $\hat{e} : \gamma$, its result will be independent of γ 's respective representation type, as well as of the concrete value $\hat{e}$. Formally, we can phrase the following property:

THEOREM 4 (OPACITY). *Let e be an expression with $\alpha, x:\alpha \vdash e : \tau$. Assume a set of values $\hat{e}_i$ ($i = 1, \dots, n$) such that $\gamma_i \approx \tau_i \vdash \hat{e}_i : \gamma_i$. Let $\sigma_i = [\alpha := \gamma_i, x := \hat{e}_i]$ for all i . If $e\sigma_1$ is not a value then there is an e' with $\alpha, x:\alpha \vdash e' : \tau$ such that $e\sigma_i \rightarrow e'\sigma_i$ for all σ_i .*

Opacity subsumes *value abstraction* [10], but is slightly stronger because it also implies *type abstraction*, i.e. independence from abstract type representations.

4.5 Sharing

Despite opacity, λ_N still allows expressing (dynamic) *sharing* between abstract types. For example, the following function checks if two given complex types are compatible and mixes operations from both of them if that is the case:

$$\begin{aligned}
& \lambda C_1 : \text{COMPLEX}. \lambda C_2 : \text{COMPLEX}. \\
& \text{open } \langle \alpha_1, (mk_1, \neg, \neg, _) \rangle = C_1 \text{ in} \\
& \text{open } \langle \alpha_2, (\neg, \neg, im_2, _) \rangle = C_2 \text{ in} \\
& \text{tcase } im_2 : \alpha_2 \rightarrow \text{real} \text{ of } im_2' : \alpha_1 \rightarrow \text{real} \\
& \text{then } im_2'(mk_1(0, 2)) \text{ else } \perp
\end{aligned}$$

(1)	$(\text{fix } x_1(x_2:\tau_2):\tau_1.e) \hat{e} \rightarrow e[x_1 := \text{fix } x_1(x_2:\tau_2):\tau_1.e, x_2 := \hat{e}]$	
(2)	$(\Lambda\alpha.e) \tau \rightarrow e[\alpha := \tau]$	
(3)	$\{\{\hat{e} : \tau_1\}_{\gamma \approx \tau'}^+ : \tau_2\}_{\gamma \approx \tau'}^- \rightarrow \hat{e}$	(if $\tau_1 = \tau_2 = \gamma$)
(4)	$\{\hat{e} : \tau\}_{\gamma \approx \tau'}^\pm \rightarrow \hat{e}$	(if $\tau = \gamma' \neq \gamma$)
(5)	$\{\hat{e} : \tau\}_{\gamma \approx \tau'}^\pm \rightarrow \text{fix } x_1(x_2 : \{\tau_2\}_{\gamma \approx \tau'}^\pm) : \{\tau_1\}_{\gamma \approx \tau'}^\pm$	(if $\tau = \tau_2 \rightarrow \tau_1$; $x_1, x_2 \notin \text{FV}(\hat{e})$)
(6)	$\{\hat{e} : \tau\}_{\gamma \approx \tau'}^\pm \rightarrow \Lambda\alpha.\{\hat{e} \alpha : \tau_1\}_{\gamma \approx \tau'}^\pm$	(if $\tau = \forall\alpha.\tau_1$)
(7)	$\text{tcase } \hat{e} : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } e_3 \rightarrow e_2[x := \hat{e}]$	(if $\tau_1 = \tau_2$)
(8)	$\text{tcase } \hat{e} : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } e_3 \rightarrow e_3$	(if $\tau_1 \neq \tau_2$)
(9)	$(N\gamma \approx \tau.\hat{e}) e \rightarrow N\gamma \approx \tau.\hat{e} e$	($\gamma \notin \text{FTN}(e)$)
(10)	$\hat{e} (N\gamma \approx \tau.\hat{e}) \rightarrow N\gamma \approx \tau.\hat{e} \hat{e}$	($\gamma \notin \text{FTN}(\hat{e})$)
(11)	$(N\gamma \approx \tau.\hat{e}) \tau' \rightarrow N\gamma \approx \tau.\hat{e} \tau'$	($\gamma \notin \text{FTN}(\tau')$)
(12)	$\{N\gamma \approx \tau.\hat{e} : \tau''\}_{\gamma' \approx \tau'}^\pm \rightarrow N\gamma \approx \tau.\{\hat{e} : \tau''\}_{\gamma' \approx \tau'}^\pm$	($\gamma \neq \gamma'; \gamma \notin \text{FTN}(\tau', \tau'')$)
(13)	$\text{tcase } N\gamma \approx \tau.\hat{e}_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } e_3 \rightarrow N\gamma \approx \tau.\text{tcase } \hat{e}_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } e_3$	($\gamma \notin \text{FTN}(\tau_1, \tau_2, e_2, e_3)$)
(14)	$e e_2 \rightarrow e' e_2$	(if $e \rightarrow e'$)
(15)	$\hat{e} e \rightarrow \hat{e} e'$	(if $e \rightarrow e'$)
(16)	$e \tau \rightarrow e' \tau$	(if $e \rightarrow e'$)
(17)	$N\gamma \approx \tau.e \rightarrow N\gamma \approx \tau.e'$	(if $e \rightarrow e'$)
(18)	$\{e : \tau'\}_{\gamma \approx \tau}^\pm \rightarrow \{e' : \tau'\}_{\gamma \approx \tau}^\pm$	(if $e \rightarrow e'$)
(19)	$\text{tcase } e : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } e_3 \rightarrow \text{tcase } e' : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } e_3$	(if $e \rightarrow e'$)

Figure 5: λ_N Reduction

That is, we are able to find out dynamically whether abstract types are compatible, although we cannot look at their representation. This ability is important for dynamic programming with abstract types. For example, consider a scenario where a process retrieves values of abstract types from different, statically undetermined locations. In order to combine those values, the program must be able to dynamically verify their compatibility.

5. HIGHER-ORDER TYPES

The λ_N -calculus is equipped with a second-order type system. That enables it to model generativity of proper types. However, many programming languages allow the definition of “polymorphic” abstract types. E.g. in SML, we can define a polymorphic abstract stack:

```
signature STACK =
sig
  type 'a stack
  val empty : 'a stack
  val push : 'a * 'a stack -> 'a stack
  val pop : 'a stack -> 'a * 'a stack
end
structure Stack :> STACK =
struct
  type 'a stack = 'a list
  ...
end
```

Such higher-order type abstraction can be captured by extending the calculus with higher-order types, allowing to

define

$$N\gamma \approx (\lambda\alpha:\Omega.\text{list } \alpha).e$$

(assuming existence of a type constructor $\text{list} : \Omega \rightarrow \Omega$). The essentials of the higher-order calculus are shown in figure 8. Besides the standard modifications to typing rules that are necessary when moving from System F to F^ω (which have been omitted for space reasons) [8, 20], typing has to deal with higher-kinded type names. As a minor technicality, we also add a kind annotation to N-binders that is not strictly necessary in λ_N^ω per se, but needed for the extension presented in the next section. We will omit this annotation where obvious. The type equivalence relation (omitted) has to be extended with β and η -rules as well as obvious rules for pushing unsealed types through type abstraction and application. Values of abstract type no longer need to have plain type γ , but generally have a type of the form $\gamma \tau_1 \dots \tau_n$ (with $n \geq 0$), which we abbreviate as $\gamma \bar{\tau}$. For example, in an encoding of the stack abstraction, integer stack values have shape $\{\hat{e} : \gamma \text{int}\}_{\gamma \approx \lambda\alpha:\Omega.\text{list } \alpha}^+ : \gamma \text{int}$.

The primary complication in the higher-order extension appears with the reduction rules for coercions: an abstract type may have the form $\gamma_1 \bar{\tau}$, and some γ_2 may appear in $\bar{\tau}$. For example, consider an expression

$$\{\hat{e} : \gamma_1 \gamma_2\}_{\gamma_2 \approx \tau_2}^-$$

How can the coercion be pushed inward, across the unrelated abstract type γ_1 ? Fortunately, the canonical forms lemma for the calculus implies $\hat{e} \equiv \{\hat{e}' : \gamma_1 \gamma_2\}_{\gamma_1 \approx \tau_1}^+$. We can hence exchange both coercions, yielding simpler ones that can be

$$\begin{array}{c}
\frac{}{\vdash \cdot : \diamond} \quad \frac{\vdash \Gamma : \diamond \quad \Gamma \vdash \tau : \Omega}{\vdash \Gamma, x : \tau : \diamond} (x \notin \text{Dom}(\Gamma)) \\
\\
\frac{\vdash \Gamma : \diamond}{\vdash \Gamma, \alpha : \diamond} (\alpha \notin \text{Dom}(\Gamma)) \quad \frac{\vdash \Gamma : \diamond \quad \Gamma \vdash \tau : \Omega}{\vdash \Gamma, \gamma \approx \tau : \diamond} (\gamma \notin \text{Dom}(\Gamma)) \quad \frac{\tau_1 = \tau'_1 \quad \tau_2 = \tau'_2}{\tau_1 \rightarrow \tau_2 = \tau'_1 \rightarrow \tau'_2} \quad \frac{\tau = \tau'}{\forall \alpha. \tau = \forall \alpha. \tau'} \quad \frac{\tau' = \tau''}{\tau = \tau''} \\
\\
\frac{\vdash \Gamma : \diamond \quad \alpha \in \Gamma}{\Gamma \vdash \alpha : \Omega} \quad \frac{\vdash \Gamma : \diamond \quad \gamma \approx \tau \in \Gamma}{\Gamma \vdash \gamma : \Omega} \quad \frac{\Gamma \vdash \tau_1 : \Omega \quad \Gamma \vdash \tau_2 : \Omega}{\Gamma \vdash \tau_1 \rightarrow \tau_2 : \Omega} \\
\\
\frac{\Gamma, \alpha \vdash \tau : \Omega}{\Gamma \vdash \forall \alpha. \tau : \Omega} \quad \frac{\Gamma \vdash \tau_1 : \Omega \quad \gamma \approx \tau_2 \in \Gamma}{\Gamma \vdash \{\tau_1\}_{\gamma \approx \tau_2} : \Omega} \\
\\
(\text{ID}) \frac{\vdash \Gamma : \diamond \quad x : \tau \in \Gamma}{\Gamma \vdash x : \tau} \quad (\text{APP}) \frac{\Gamma \vdash e_1 : \tau' \rightarrow \tau \quad \Gamma \vdash e_2 : \tau'}{\Gamma \vdash e_1 e_2 : \tau} \\
\\
(\text{FIX}) \frac{\Gamma \vdash \tau_1 : \Omega \quad \Gamma \vdash \tau_2 : \Omega \quad \Gamma, x_1 : \tau_2 \rightarrow \tau_1, x_2 : \tau_2 \vdash e : \tau_1}{\Gamma \vdash (\text{fix } x_1 (x_2 : \tau_2) : \tau_1. e) : \tau_2 \rightarrow \tau_1} \\
\\
(\text{GEN}) \frac{\Gamma, \alpha \vdash e : \tau}{\Gamma \vdash \Lambda \alpha. e : \forall \alpha. \tau} \quad (\text{INST}) \frac{\Gamma \vdash e : \forall \alpha. \tau \quad \Gamma \vdash \tau' : \Omega}{\Gamma \vdash e \tau' : \tau[\alpha := \tau']} \\
\\
(\text{NEW}) \frac{\Gamma \vdash \tau' : \Omega \quad \Gamma, \gamma \approx \tau' \vdash e : \tau}{\Gamma \vdash N \gamma \approx \tau'. e : \tau} (\gamma \notin \text{FTN}(\tau)) \\
\\
(\text{SEAL}) \frac{\Gamma \vdash e : \{\tau\}_{\gamma \approx \tau'}^- \quad \gamma \approx \tau' \in \Gamma}{\Gamma \vdash \{e : \tau\}_{\gamma \approx \tau'}^+ : \tau} \\
\\
(\text{UNSEAL}) \frac{\Gamma \vdash e : \tau \quad \gamma \approx \tau' \in \Gamma}{\Gamma \vdash \{e : \tau\}_{\gamma \approx \tau'}^- : \{\tau\}_{\gamma \approx \tau'}^-} \\
\\
(\text{TCASE}) \frac{\Gamma \vdash e_1 : \tau_1 \quad \Gamma \vdash \tau_2 : \Omega \quad \Gamma, x : \tau_2 \vdash e_2 : \tau \quad \Gamma \vdash e_3 : \tau}{\Gamma \vdash (\text{tcase } e_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } e_3) : \tau} \\
\\
(\text{EQUIV}) \frac{\Gamma \vdash e : \tau' \quad \tau' = \tau \quad \Gamma \vdash \tau : \Omega}{\Gamma \vdash e : \tau}
\end{array}$$

Figure 6: λ_N Typing

coped with as usual (or by another step of the same sort):

$$\{\{\hat{e}' : \gamma_1 \gamma_2\}_{\gamma_1}^+ : \gamma_1 \gamma_2\}_{\gamma_2}^- \rightarrow \{\{\hat{e}' : \tau_1 \gamma_2\}_{\gamma_2}^- : \gamma_1 \tau_2\}_{\gamma_1}^+$$

In general however, γ_1 can occur in τ_2 , or γ_2 may occur in τ_1 (since type assertions cannot be circular, at most one of these cases can actually arise at a time). Either way, the reduct would not be well-typed. We hence have to insert an auxiliary coercion, reducing to either

$$\{\{\{\hat{e}' : \tau_1 \gamma_2\}_{\gamma_2}^- : \tau_1 \tau_2\}_{\gamma_1}^- : \gamma_1 \tau_2\}_{\gamma_1}^+ \quad (\gamma_2 \notin \text{FTN}(\tau_1))$$

or

$$\{\{\{\hat{e}' : \tau_1 \gamma_2\}_{\gamma_2}^- : \tau_1 \{\tau_2\}_{\gamma_1}^- : \gamma_1 \tau_2\}_{\gamma_1}^+ : \gamma_1 \tau_2\}_{\gamma_1}^+ \quad (\gamma_1 \notin \text{FTN}(\tau_2))$$

depending on which case actually applies. In a similar vein, positive coercions have to be handled. The modified reduction rules for higher-order coercions are shown in figure 8. To keep side conditions readable we use the following conventions: (1) $\gamma \neq \gamma'$; (2) $\{\hat{e} : \tau'\}_{\gamma \approx \tau}^\pm$ matches any term $\{\hat{e} : \tau''\}_{\gamma \approx \tau}^\pm$ with $\tau'' = \tau'$ and $\cdot \vdash \tau'' : \kappa$; (3) $\gamma \in \text{FTN}(\tau)$

$$\begin{array}{c}
\frac{}{\tau = \tau} \quad \frac{\tau' = \tau}{\tau = \tau'} \quad \frac{\tau = \tau' \quad \tau' = \tau''}{\tau = \tau''} \\
\\
\frac{\tau_1 = \tau'_1 \quad \tau_2 = \tau'_2}{\tau_1 \rightarrow \tau_2 = \tau'_1 \rightarrow \tau'_2} \quad \frac{\tau = \tau'}{\forall \alpha. \tau = \forall \alpha. \tau'} \quad \frac{\tau_1 = \tau'_1}{\{\tau_1\}_{\gamma \approx \tau_2} = \{\tau'_1\}_{\gamma \approx \tau_2}} \\
\\
\frac{}{\{\gamma\}_{\gamma \approx \tau} = \tau} \quad \frac{}{\{\gamma'\}_{\gamma \approx \tau} = \gamma'} (\gamma \neq \gamma') \\
\\
\frac{}{\{\tau_1 \rightarrow \tau_2\}_{\gamma \approx \tau_3} = \{\tau_1\}_{\gamma \approx \tau_3} \rightarrow \{\tau_2\}_{\gamma \approx \tau_3}} \\
\\
\frac{}{\{\forall \alpha. \tau_1\}_{\gamma \approx \tau_2} = \forall \alpha. \{\tau_1\}_{\gamma \approx \tau_2}} (\alpha \notin \text{FTV}(\tau_2))
\end{array}$$

Figure 7: λ_N Type equivalence

means $\forall \tau'. \tau' = \tau \Rightarrow \gamma \in \text{FTN}(\tau')$. Reduction rule (4a) generalizes the previous rule (4), while rules (4b)–(4e) treat the cases discussed above. They also handle γ' occurring in its own argument vector $\vec{\tau}$.

Soundness results extend to λ_N^ω in a straight-forward manner. Opacity has to be reformulated as follows:

THEOREM 5 (λ_N^ω OPACITY). *Let e be an expression with $\alpha, x : \alpha \vec{\tau} \vdash e : \tau$ for some $\vec{\tau}$. Assume a set of values $\hat{e}_i$ ($i = 1, \dots, n$) such that $\gamma_i \approx \tau_i \vdash \hat{e}_i : \gamma_i \vec{\tau}$. Let $\sigma_i = [\alpha := \gamma_i, x := \hat{e}_i]$ for all i . If $e \sigma_1$ is not a value then there is an e' with $\alpha, x : \alpha \vec{\tau} \vdash e' : \tau$ such that $e \sigma_i \rightarrow e' \sigma_i$ for all σ_i .*

5.1 Applicative generativity

A function containing a N-binder will produce a new copy of the binder on every application. For example,

$$\begin{aligned}
&\text{let } f = \lambda x : \text{unit}. N \gamma \approx \text{int}. \langle \gamma, \{13\}_{\gamma}^+ \rangle \text{ in } (f \ (), f \ ()) \\
&= N \gamma_1 \approx \text{int}. N \gamma_2 \approx \text{int}. (\langle \gamma_1, \{13\}_{\gamma_1}^+ \rangle, \langle \gamma_2, \{13\}_{\gamma_2}^+ \rangle)
\end{aligned}$$

The standard α -renaming rules make γ_1 and γ_2 two incompatible types. That behaviour is analogous to functors in SML, where the following snippet declares two incompatible types **X1.t** and **X2.t**:

```

functor F () :> sig type t; val x : t end
= struct type t = int; val x = 13 end
structure X1 = F ()
structure X2 = F ()

```

In other words, λ_N implements a *fully generative* type abstraction discipline. There are alternative approaches to functors that make **X1.t** and **X2.t** equivalent types. After Leroy, such functors are called *applicative* [12]. Dreyer, Crary and Harper propose two alternative sealing operators $>$ and $>>$ to allow generative and applicative functors to co-exist [6]. In their approach, replacing $>$ by $>>$ in the above example would yield compatible types **X1.t** and **X2.t**.

We can incorporate applicative generativity into λ_N^ω by extending it with a second form of N-binder, which we distinguish by marking it as follows:

$$\check{N} \gamma : \kappa \approx \tau. e$$

Typing for this binder is the same as for plain N, but it comes with different reduction rules — the fundamental idea being that it is lifted out of lambdas prior to β -reduction, avoiding

(kinds)	$\kappa ::= \Omega \mid \kappa_1 \rightarrow \kappa_2$	$\frac{\gamma \approx \tau \in \Gamma \quad \Gamma \vdash \tau : \kappa}{\Gamma \vdash \gamma : \kappa}$
(types)	$\tau ::= \dots \mid \forall \alpha : \kappa. \tau \mid \lambda \alpha : \kappa. \tau \mid \tau_1 \tau_2$	
(terms)	$e ::= \dots \mid \Lambda \alpha : \kappa. e \mid N \gamma : \kappa \approx \tau. e$	
(values)	$\hat{e} ::= \dots \mid \Lambda \alpha : \kappa. e \mid \{\hat{e} : \tau\}_{\gamma \approx \tau'}^+ \ (\tau = \gamma \bar{\tau})$	
(env's)	$\Gamma ::= \dots \mid \Gamma, \alpha : \kappa$	
		(NEW) $\frac{\Gamma \vdash \tau' : \kappa \quad \Gamma, \gamma \approx \tau' \vdash e : \tau}{\Gamma \vdash N \gamma : \kappa \approx \tau'. e : \tau} (\gamma \notin \text{FTN}(\tau))$
(3)	$\{\{\hat{e} : \gamma \bar{\tau}\}_{\gamma \approx \tau}^+ : \gamma \bar{\tau}\}_{\gamma \approx \tau}^- \rightarrow \hat{e}$	
(4a)	$\{\{\hat{e} : \gamma' \bar{\tau}'\}_{\gamma' \approx \tau'}^+ : \gamma' \bar{\tau}'\}_{\gamma \approx \tau}^\pm \rightarrow \{\hat{e} : \gamma' \bar{\tau}'\}_{\gamma'}^\pm$	(if $\gamma \notin \text{FTN}(\bar{\tau})$)
(4b)	$\{\{\hat{e} : \gamma' \bar{\tau}'\}_{\gamma' \approx \tau'}^+ : \gamma' \bar{\tau}'\}_{\gamma \approx \tau}^+ \rightarrow \{\{\{\hat{e} : \tau' \{\{\bar{\tau}\}_{\gamma'}^-\}_{\gamma'}^-\} : \tau' \{\{\bar{\tau}\}_{\gamma'}^-\}_{\gamma'}^+\} : \gamma' \bar{\tau}'\}_{\gamma'}^+$	(if $\gamma' \notin \text{FTN}(\tau); \gamma \in \text{FTN}(\bar{\tau})$)
(4c)	$\{\{\hat{e} : \gamma' \bar{\tau}'\}_{\gamma' \approx \tau'}^+ : \gamma' \bar{\tau}'\}_{\gamma \approx \tau}^- \rightarrow \{\{\{\hat{e} : \tau' \{\{\bar{\tau}\}_{\gamma'}^-\}_{\gamma'}^-\} : \tau' \{\{\bar{\tau}\}_{\gamma'}^-\}_{\gamma'}^+\} : \gamma' \bar{\tau}'\}_{\gamma'}^+$	(if $\gamma' \notin \text{FTN}(\tau); \gamma \in \text{FTN}(\bar{\tau})$)
(4d)	$\{\{\hat{e} : \gamma' \bar{\tau}'\}_{\gamma' \approx \tau'}^+ : \gamma' \bar{\tau}'\}_{\gamma \approx \tau}^+ \rightarrow \{\{\{\hat{e} : \tau' \{\{\bar{\tau}\}_{\gamma'}^-\}_{\gamma'}^-\} : \tau' \{\{\bar{\tau}\}_{\gamma'}^-\}_{\gamma'}^+\} : \gamma' \bar{\tau}'\}_{\gamma'}^+$	(if $\gamma' \in \text{FTN}(\tau); \gamma \in \text{FTN}(\bar{\tau})$)
(4e)	$\{\{\hat{e} : \gamma' \bar{\tau}'\}_{\gamma' \approx \tau'}^+ : \gamma' \bar{\tau}'\}_{\gamma \approx \tau}^- \rightarrow \{\{\{\hat{e} : \tau' \{\{\bar{\tau}\}_{\gamma'}^-\}_{\gamma'}^-\} : \tau' \{\{\bar{\tau}\}_{\gamma'}^-\}_{\gamma'}^+\} : \gamma' \bar{\tau}'\}_{\gamma'}^+$	(if $\gamma' \in \text{FTN}(\tau); \gamma \in \text{FTN}(\bar{\tau})$)

Figure 8: The λ_N^ω -calculus (excerpt)

duplication. Figure 9 shows the extended syntax. For reasons that will become apparent shortly, we call the class of terms extended with $\tilde{N}$ *pre-terms*. Again, we need an additional class of results, called *pre-results*, to get a deterministic evaluation relation. Pre-results are ordinary λ_N^ω -terms prefixed by a sequence of $\tilde{N}$ -binders.

Figure 10 reveals the extended reduction relation. It contains scope extrusion rules for $\tilde{N}$ (20–29), structural rules on pre-terms (30–40), and a single *fixation* rule (41). Unlike plain N , the scope of $\tilde{N}$ can be extruded across binders like fix , Λ and plain N , as well as from the branches of a typecase. Moreover, pre-term reduction may proceed under all these constructs. The only interesting rules are (23) and (25), where a $\tilde{N}$ -binder has to be lifted out of a binder for a type variable. Since that variable may occur free in the respective representation type, special care has to be taken. We borrow an idea from Russo [27], who models applicative functors by representing abstract types in their result signature as higher-order abstractions over all types of the functor’s argument. In a similar vein, the aforementioned reduction rules raise the type name γ to higher order by abstracting over the respective type variable whose binding the scope will be extruded from. As a simple example, consider the following reduction:

$$\begin{aligned}
& \text{let } f = \Lambda \alpha : \Omega. \lambda x : \alpha. \tilde{N} \gamma \approx \alpha. \langle \gamma, \{x : \gamma\}_{\gamma \approx \alpha}^+ \rangle \\
& \text{in } (f \text{ int } 3, f \text{ int } 4, f \text{ real } 5.0) \\
= & \text{let } f = \tilde{N} \gamma \approx (\lambda \alpha : \Omega. \alpha). \Lambda \alpha : \Omega. \lambda x : \alpha. \langle \gamma \alpha, \{x : \gamma \alpha\}_{\gamma \approx \lambda \alpha : \Omega. \alpha}^+ \rangle \\
& \text{in } (f \text{ int } 3, f \text{ int } 4, f \text{ real } 5.0) \\
= & \tilde{N} \gamma \approx (\lambda \alpha : \Omega. \alpha). \text{let } f = \Lambda \alpha : \Omega. \lambda x : \alpha. \langle \gamma \alpha, \{x : \gamma \alpha\}_{\gamma \approx \lambda \alpha : \Omega. \alpha}^+ \rangle \\
& \text{in } (f \text{ int } 3, f \text{ int } 4, f \text{ real } 5.0) \\
= & N \gamma \approx (\lambda \alpha : \Omega. \alpha). (\langle \gamma \text{ int}, \{3\}_{\gamma}^+ \rangle, \langle \gamma \text{ int}, \{4\}_{\gamma}^+ \rangle, \langle \gamma \text{ real}, \{5.0\}_{\gamma}^+ \rangle)
\end{aligned}$$

The first two packages carry the same type $\gamma \text{ int}$, while the last one contains the different type $\gamma \text{ real}$.

Since scope extrusion is the only actual evaluation taking place on non-plain pre-terms, the effect of reducing pre-terms is lifting out all $\tilde{N}$ -binders until the pre-term has become a pre-result, i.e. its body is a plain term. Once that form has been reached, the fixation rule (41) turns all its now outermost $\tilde{N}$ -binders into plain N -binders, leaving an ordinary λ_N^ω -term, for which evaluation proceeds as before. In other words, evaluation happens in two phases: first, rules (20–41) transform the pre-term into a plain term, then the

(pre-terms)	$\tilde{e} ::= x \mid \text{fix } x_1(x_2 : \tau_2) : \tau_1. \tilde{e} \mid \tilde{e}_1 \tilde{e}_2 \mid \Lambda \alpha : \kappa. \tilde{e} \mid \tilde{e} \tau \mid N \gamma : \kappa \approx \tau. \tilde{e} \mid \tilde{N} \gamma : \kappa \approx \tau. \tilde{e} \mid \{\tilde{e} : \tau\}_{\gamma \approx \tau'}^\pm \mid \text{tcase } \tilde{e}_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } \tilde{e}_2 \text{ else } \tilde{e}_3$
(pre-results)	$\tilde{e} ::= e \mid \tilde{N} \gamma : \kappa \approx \tau. \tilde{e}$

Figure 9: Syntax of pre-terms

rules (1–19) perform proper evaluation. The first phase will always terminate:

THEOREM 6 (FINITE PRE-TERM REDUCTION). *Every well-typed pre-term $\tilde{e}$ reduces to a plain term e by a finite reduction sequence (involving only rules (20)–(41)).*

In this light, it is valid to view pre-terms as an external language, which is transformed into the internal core language of plain terms prior to evaluation via a static elaboration process.

6. RELATED WORK

Although being relatively simple in spirit, to our knowledge there is no previous work that isolates the dynamic aspect of type generativity for abstraction and formalises it in a calculus. While module theories usually account for generativity as well, they do so solely on the static level of typing rules. In fact, all of the influential theories for ML modules [12, 15, 27, 6] are not full calculi, but merely type systems, that side-step the issue of reduction. The presence of ad-hoc typing rules encompassing type abstraction precludes a type-preserving reduction semantics.

One notable exception is Sewell, who uses generativity for modelling certain aspects of type abstraction [29]. However, in his system generated abstract types are recorded as manifestly equal to their representation in a global environment, so that opacity is not properly maintained dynamically.

Glew presented a calculus for generating new tagged types at runtime and dispatching on them [9]. His system is more complex than ours in order to allow for hierarchical types, but it is not fully reflexive since untagged types cannot be analysed.

The work most relevant to ours is by Grossman, Morrisett and Zdancewic on proof techniques for abstraction

$$\begin{array}{lll}
(20) & \text{fix } x_1(x_2:\tau_2):\tau_1.\check{N}\gamma:k\approx\tau.\check{e} & \rightarrow \check{N}\gamma:k\approx\tau.\text{fix } x_1(x_2:\tau_2):\tau_1.\check{e} & (\gamma \notin \text{FTN}(\tau_1, \tau_2)) \\
(21) & (\check{N}\gamma:k\approx\tau.\check{e})\check{e} & \rightarrow \check{N}\gamma:k\approx\tau.\check{e}\check{e} & (\gamma \notin \text{FTN}(\check{e})) \\
(22) & e(\check{N}\gamma:k\approx\tau.\check{e}) & \rightarrow \check{N}\gamma:k\approx\tau.e\check{e} & (\gamma \notin \text{FTN}(e)) \\
(23) & \Lambda\alpha:\kappa'.\check{N}\gamma:k\approx\tau.\check{e} & \rightarrow \check{N}\gamma:\kappa' \rightarrow \kappa\approx(\lambda\alpha:\kappa'.\tau).\Lambda\alpha:\kappa'.\check{e}[\gamma := \gamma \alpha] & \\
(24) & (\check{N}\gamma:k\approx\tau.\check{e})\tau' & \rightarrow \check{N}\gamma:k\approx\tau.\check{e}\tau' & (\gamma \notin \text{FTN}(\tau')) \\
(25) & N\gamma':\kappa' \approx \tau'.\check{N}\gamma:k\approx\tau.\check{e} & \rightarrow \check{N}\gamma:\kappa' \rightarrow \kappa\approx(\lambda\alpha:\kappa'.\tau[\gamma' := \alpha]).N\gamma':\kappa' \approx \tau'.\check{e}[\gamma := \gamma \gamma'] & (\gamma \neq \gamma'; \alpha \notin \text{FTV}(\tau); \gamma \notin \text{FTN}(\tau')) \\
(26) & \{\check{N}\gamma:k\approx\tau.\check{e} : \tau''\}_{\gamma' \approx \tau'}^\pm & \rightarrow \check{N}\gamma:k\approx\tau.\{\check{e} : \tau''\}_{\gamma' \approx \tau'}^\pm & (\gamma \neq \gamma'; \gamma \notin \text{FTN}(\tau', \tau'')) \\
(27) & \text{tcase } \check{N}\gamma:k\approx\tau.\check{e}_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } \check{e}_2 \text{ else } \check{e}_3 & \rightarrow \check{N}\gamma:k\approx\tau.\text{tcase } \check{e}_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } \check{e}_2 \text{ else } \check{e}_3 & (\gamma \notin \text{FTN}(\tau_1, \tau_2, \check{e}_2, \check{e}_3)) \\
(28) & \text{tcase } e_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } \check{N}\gamma:k\approx\tau.\check{e}_2 \text{ else } \check{e}_3 & \rightarrow \check{N}\gamma:k\approx\tau.\text{tcase } e_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } \check{e}_2 \text{ else } \check{e}_3 & (\gamma \notin \text{FTN}(\tau_1, \tau_2, e_1, \check{e}_3)) \\
(29) & \text{tcase } e_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } \check{N}\gamma:k\approx\tau.\check{e}_3 & \rightarrow \check{N}\gamma:k\approx\tau.\text{tcase } e_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } \check{e}_3 & (\gamma \notin \text{FTN}(\tau_1, \tau_2, e_1, e_2)) \\
\\
(30) & \text{fix } x_1(x_2:\tau_2):\tau_1.\check{e} & \rightarrow \text{fix } x_1(x_2:\tau_2):\tau_1.\check{e}' & (\text{if } \check{e} \rightarrow \check{e}'; \check{e} \neq \check{e}') \\
(31) & \check{e}\check{e}_2 & \rightarrow \check{e}'\check{e}_2 & (\text{if } \check{e} \rightarrow \check{e}'; \check{e} \neq \check{e}') \\
(32) & e\check{e} & \rightarrow e\check{e}' & (\text{if } \check{e} \rightarrow \check{e}'; \check{e} \neq \check{e}') \\
(33) & \Lambda\alpha:\kappa.\check{e} & \rightarrow \Lambda\alpha:\kappa.\check{e}' & (\text{if } \check{e} \rightarrow \check{e}'; \check{e} \neq \check{e}') \\
(34) & \check{e}\tau & \rightarrow \check{e}'\tau & (\text{if } \check{e} \rightarrow \check{e}'; \check{e} \neq \check{e}') \\
(35) & N\gamma:k\approx\tau.\check{e} & \rightarrow N\gamma:k\approx\tau.\check{e}' & (\text{if } \check{e} \rightarrow \check{e}'; \check{e} \neq \check{e}') \\
(36) & \check{N}\gamma:k\approx\tau.\check{e} & \rightarrow \check{N}\gamma:k\approx\tau.\check{e}' & (\text{if } \check{e} \rightarrow \check{e}'; \check{e} \neq \check{e}') \\
(37) & \{\check{e} : \tau'\}_{\gamma \approx \tau}^\pm & \rightarrow \{\check{e}' : \tau'\}_{\gamma \approx \tau}^\pm & (\text{if } \check{e} \rightarrow \check{e}'; \check{e} \neq \check{e}') \\
(38) & \text{tcase } \check{e} : \tau_1 \text{ of } x : \tau_2 \text{ then } \check{e}_2 \text{ else } \check{e}_3 & \rightarrow \text{tcase } \check{e}' : \tau_1 \text{ of } x : \tau_2 \text{ then } \check{e}_2 \text{ else } \check{e}_3 & (\text{if } \check{e} \rightarrow \check{e}'; \check{e} \neq \check{e}') \\
(39) & \text{tcase } e_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } \check{e} \text{ else } \check{e}_3 & \rightarrow \text{tcase } e_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } \check{e}' \text{ else } \check{e}_3 & (\text{if } \check{e} \rightarrow \check{e}'; \check{e} \neq \check{e}') \\
(40) & \text{tcase } e_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } \check{e} & \rightarrow \text{tcase } e_1 : \tau_1 \text{ of } x : \tau_2 \text{ then } e_2 \text{ else } \check{e}' & (\text{if } \check{e} \rightarrow \check{e}'; \check{e} \neq \check{e}') \\
\\
(41) & \check{N}\gamma:k\approx\tau.e & \rightarrow N\gamma:k\approx\tau.e &
\end{array}$$

Figure 10: Reduction for applicative generation

[10]. They present a calculus that uses annotated brackets for marking abstraction boundaries during reduction. These are similar to the generalized coercions in λ_N . However, in their system abstraction brackets are not directed, i.e. it does not distinguish between sealing and unsealing. Instead, all directly nested brackets are collapsed on reduction and annotated with the sequence of ‘principals’ that own the corresponding abstractions. That appears to be slightly more complex, but avoids the need for the artefact of unsealed types, as well as η -expansion during reduction. The latter is advantageous for proving a type erasure theorem. On the other hand, in their system the definition of type equivalence depends on an additional type assertion environment. This complicates the operational semantics, because the environment has to be maintained dynamically to cope with abstraction scoping. Furthermore, their calculus cannot express dynamic abstraction, but requires identifying a fixed set of abstractions statically, since technically, the reduction relation has to be extended for each occurring abstraction. Both these aspects make it less suited as a simple operational model for type abstraction.

The λ_N -calculus also reveals close similarities to Pierce and Sumii’s cryptographic lambda calculus [21]: N-binders correspond to key generation and sealing/unsealing to encryption/decryption operations in that calculus. However, their type system is weaker in the sense that decryption may fail dynamically. They present an encoding of type abstracting polymorphism using ciphertext, but do not prove anything about it.

None of the mentioned work considers higher-order types and higher-order sealing, or applicative generativity.

7. CONCLUSION

The standard encoding of abstract types via existential types relies on parametricity of polymorphism. If parametricity is not given, due to constructs for type analysis, the encoding is inappropriate because it cannot warrant encapsulation. In non-parametric settings it is necessary to capture generativity to achieve dynamic opacity and thereby encapsulation.

As a solution we proposed a calculus whose core feature is a syntactic treatment of dynamic generativity, using a variation of name generation as known from π -calculus and other systems. It relies on coercions as explicit transition markers for abstraction boundaries. By generalizing these coercions inductively over all types they can be used to express sealing. The calculus can be extended to higher-order abstract types and augmented with support for applicative generativity.

As future work, we would like to integrate aspects of λ_N^{w} with recent module theories [6], in order to get a full theory of modules with dynamic typing. For example, the language Alice ML that is currently being developed [2] provides so-called *packages* as a form of dynamics generalized to modules. A combined theory is needed to give a formal semantics for that feature.

Full representation independence or extensionality [18, 22] of abstract types in λ_N is a challenge to prove. There appears to be very little work on proof techniques for operational equivalence in non-parametric extensions of the λ -calculus. It is not clear how techniques like logical relations [18, 25, 23] can be applied in such a setting.

8. REFERENCES

- [1] M. Abadi, L. Cardelli, B. Pierce, and D. Rémy. Dynamic typing in polymorphic languages. *Journal of Functional Programming*, 5(1):111–130, Jan. 1995.
- [2] Alice Team. *The Alice System*. Programming System Lab, Universität des Saarlandes, <http://www.ps.un-sb.de/alice/>, 2003.
- [3] H. Barendregt. Lambda calculi with types. In S. Abramsky, D. Gabbay, and T. Maibaum, editors, *Handbook of Logic in Computer Science*, volume 2, chapter 2, pages 117–309. Oxford University Press, 1992.
- [4] L. Cardelli, J. Donahue, L. Glassman, M. Jordan, B. Kalsow, and G. Nelson. Modula-3 language definition. In G. Nelson, editor, *System Programming with Modula-3*, chapter 2, pages 11–66. Prentice Hall, 1991.
- [5] L. Cardelli and X. Leroy. Abstract types and the dot notation. In *IFIP TC2 working conference on programming concepts and methods*, pages 479–504. North-Holland, Mar. 1990.
- [6] D. Dreyer, K. Crary, and R. Harper. A type system for higher-order modules. In *30th Symposium on Principles of Programming Languages*, New Orleans, USA, Jan. 2003.
- [7] C. Dubois, F. Rouaix, and P. Weis. Extensional polymorphism. In *22nd Symposium on Principles of Programming Languages*, San Francisco, USA, Jan. 1995.
- [8] J.-Y. Girard. *Interprétation Fonctionnelle et Élimination des Coupures de l'Arithmétique d'Ordre Supérieur*. PhD thesis, June 1972.
- [9] N. Glew. Type dispatch for named hierarchical types. In *International Conference on Functional Programming*, Paris, France, Oct. 1999.
- [10] D. Grossman, G. Morrisett, and S. Zdancewic. Syntactic type abstraction. *Transactions on Programming Languages and Systems*, 22(6):1037–1080, Nov. 2000.
- [11] R. Harper and G. Morrisett. Compiling polymorphism using intensional type analysis. In *22nd Symposium on Principles of Programming Languages*, pages 130–141, San Francisco, USA, Jan. 1995.
- [12] X. Leroy. Applicative functors and fully transparent higher-order modules. In *22nd Symposium on Principles of Programming Languages*, pages 142–153, San Francisco, USA, Jan. 1995. ACM.
- [13] X. Leroy. *The Objective Caml System*. INRIA, 2003. <http://pauillac.inria.fr/ocaml/htmlman/>.
- [14] X. Leroy and M. Mauny. Dynamics in ML. *Journal of Functional Programming*, 3(4):431–463, 1993.
- [15] M. Lillibridge. *Translucent Sums: A Foundation for Higher-Order Module Systems*. PhD thesis, School of Computer Science, Carnegie Mellon University, Pittsburgh, USA, May 1997.
- [16] B. Liskov, R. Atkinson, T. Bloom, E. Moss, C. Schaffert, R. Scheifler, and A. Snyder. CLU reference manual. Technical Report MIT/LCS/TR-225, 1979.
- [17] R. Milner, M. Tofte, R. Harper, and D. MacQueen. *Definition of Standard ML (Revised)*. The MIT Press, 1997.
- [18] J. Mitchell. On the equivalence of data representations. In V. Lifschitz, editor, *Artificial Intelligence and Mathematical Theory of Computation: Papers in Honor of John McCarthy*, pages 305–330. Academic Press, 1991.
- [19] J. Mitchell and G. Plotkin. Abstract types have existential type. *Transactions on Programming Languages and Systems*, 10(3):470–502, 1988. Preliminary version appeared in *12th Symposium on Principles of Programming Languages*, 1985.
- [20] B. Pierce. *Types and Programming Languages*. The MIT Press, Feb. 2002.
- [21] B. Pierce and E. Sumii. Relating cryptography and polymorphism. Technical report, July 2000. <http://www.yl.is.s.u-tokyo.ac.jp/~sumii/pub/>.
- [22] A. Pitts. Existential types: Logical relations and operational equivalence. In *25th International Colloquium on Automata, Languages and Programming*, volume 1443 of *Lecture Notes in Computer Science*, pages 309–326. Springer-Verlag, Berlin, 1998.
- [23] A. Pitts. Parametric polymorphism and operational equivalence. *Mathematical Structures in Computer Science*, 10:321–359, 2000.
- [24] A. Pitts and I. Stark. On the observable properties of higher order functions that dynamically create local names. In P. Hudak, editor, *Workshop on State in Programming Languages*, pages 31–45, Copenhagen, Denmark, 1993.
- [25] G. Plotkin and M. Abadi. A logic for parametric polymorphism. In M. Beeze and J. F. Groote, editors, *Typed Lambda Calculus and Applications*, volume 664 of *Lecture Notes in Computer Science*, pages 361–375. Springer-Verlag, Berlin, 1993.
- [26] J. Reynolds. Types, abstraction and parametric polymorphism. In R. Mason, editor, *Information Processing*, pages 513–523, Amsterdam, 1983. North Holland.
- [27] C. Russo. *Types for Modules*. Dissertation, University of Edinburgh, 1998.
- [28] D. Sangiorgi and D. Walker. *The π -calculus: a Theory of Mobile Processes*. Cambridge University Press, Dec. 2001.
- [29] P. Sewell. Modules, abstract types, and distributed versioning. In *28th Symposium on Principles of Programming Languages*, London, UK, Jan. 2001.
- [30] C. Strachey. Fundamental concepts in programming languages. In *Lecture Notes, International Summer School in Computer Programming*. Copenhagen, Aug. 1967. Reprinted in: *Higher-Order and Symbolic Computation*, 13(1–2):11–49, April 2000.
- [31] V. Trifonov, B. Saha, and Z. Shao. Fully reflexive intensional type analysis. In *Fifth International Conference on Functional Programming*, pages 82–93, Montreal, Canada, Sept. 2000.
- [32] S. Weirich. Type-safe cast. In *International Conference on Functional Programming*, pages 58–67, Montreal, Canada, Sept. 2000.
- [33] N. Wirth. *Programming in MODULA-2*. Springer-Verlag, 3rd edition, 1985.